Mastering Linked Lists

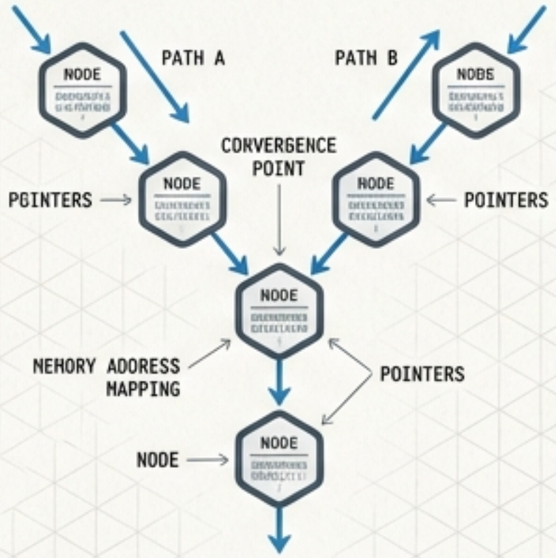
Patterns & Optimizations: A Masterclass in Pointer Arithmetic

Based on the technical breakdown of
LeetCode 160, 234, and 237 by CTO Bhaiya

The goal isn't just to solve the problem.
*The goal is to **master the pointers**.*

The Landscape of Manipulation

WIDTH: 33.3%



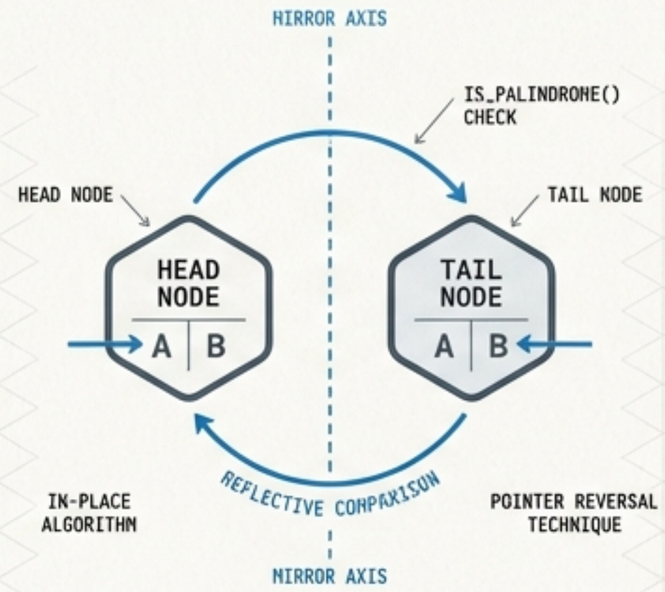
Synchronization

Finding where two independent paths converge in memory.

TIME COMPLEXITY: O(N+M)

SPACE COMPLEXITY: O(1)

WIDTH: 33.3%



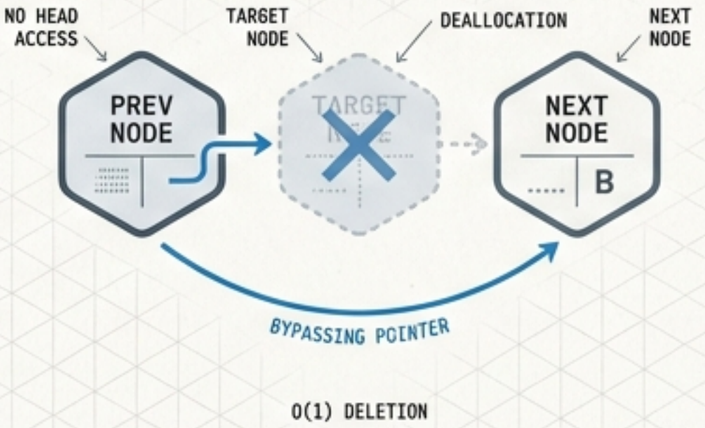
State Modification

Verifying symmetry without allocating extra space.

IN-PLACE ALGORITHM

POINTER REVERSAL TECHNIQUE

WIDTH: 33.3%



Lateral Thinking

Deleting a node without access to the head pointer.

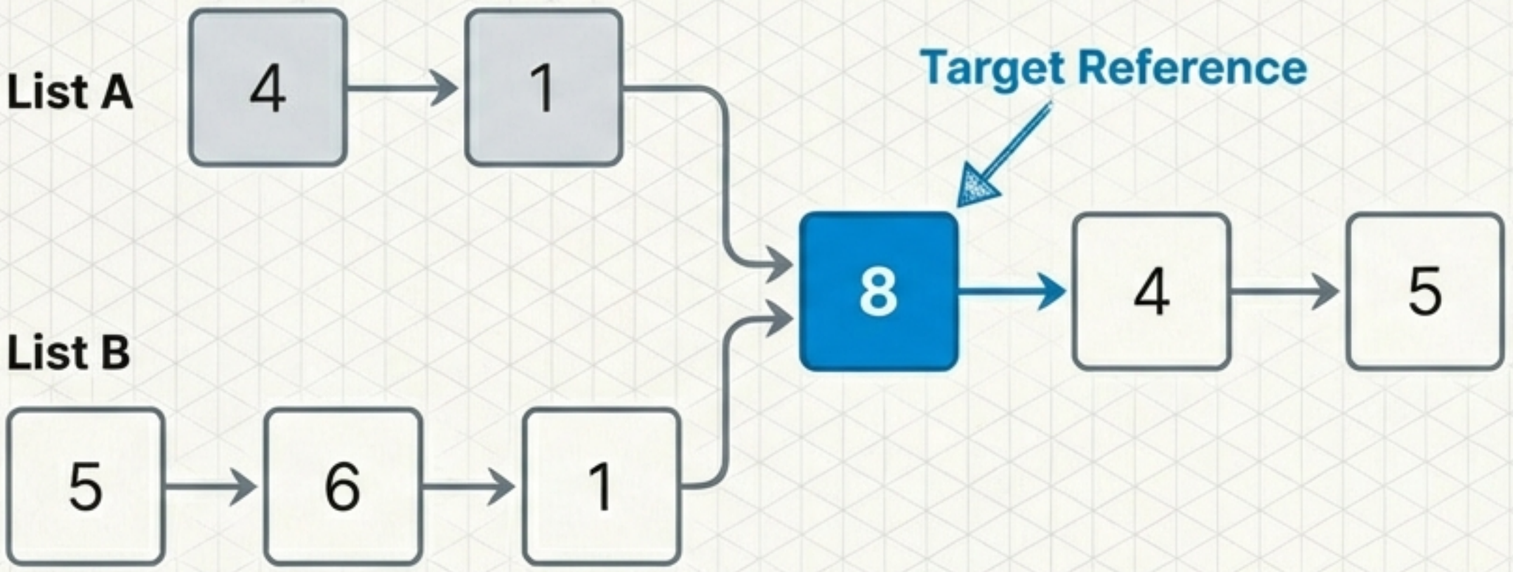
DATA OVERWRITE METHOD

POINTER MANIPULATION

WIDTH: 33.3%

Philosophy: It is not about the data inside the node. It is about the creative orchestration of pointers to save Time and Space.

The Convergence Problem (LeetCode (LeetCode 160)



The Goal

Return the reference node where List A and List B intersect.

If no intersection, return null.

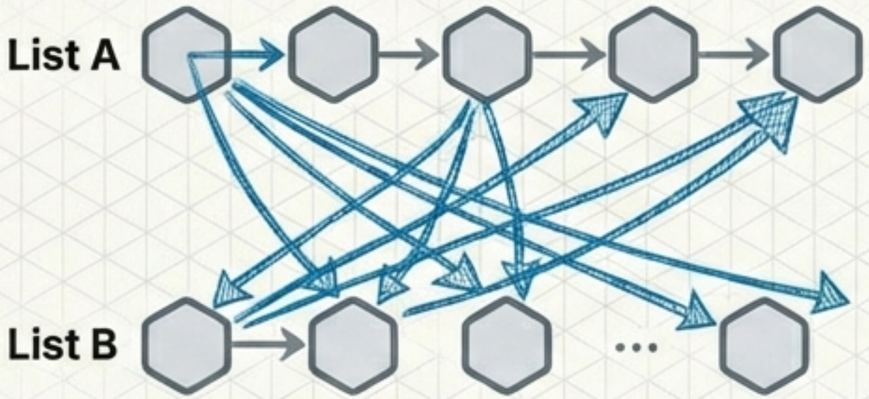
Constraints

Time Complexity: O(N) preferred

Space Complexity Requirement: O(1)

The Trade-off: Speed vs. Memory

Approach 1 (Brute Force)



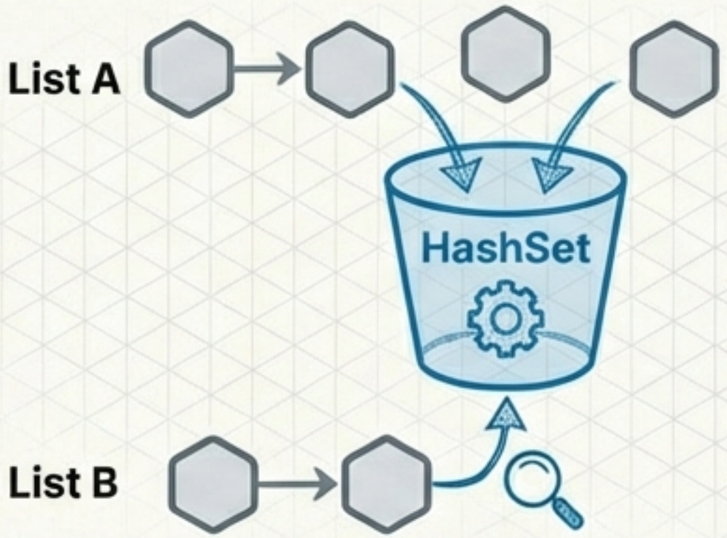
Iterate through List A. For every node, iterate through all of List B.

TC: O(M * N)

SC: O(1)

Too Slow.

Approach 2 (HashSet)



Store references of List A in a Set. Traverse List B to check for existence.

TC: O(M + N)

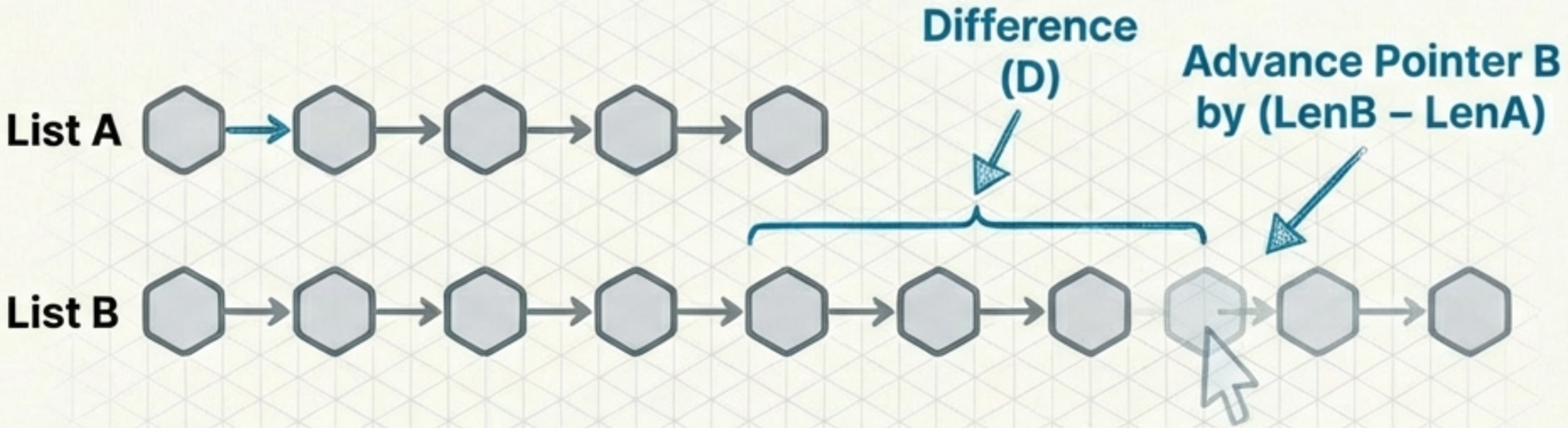
SC: O(N)

Memory Heavy. Violates Constraints.

Approach 3: Aligning the Starting Line

Algorithm Steps

1. Calculate Length A and Length B.
2. Find Difference D = |LenA - LenB|.
3. Move longer list pointer D steps.
4. Move both simultaneously until collision.

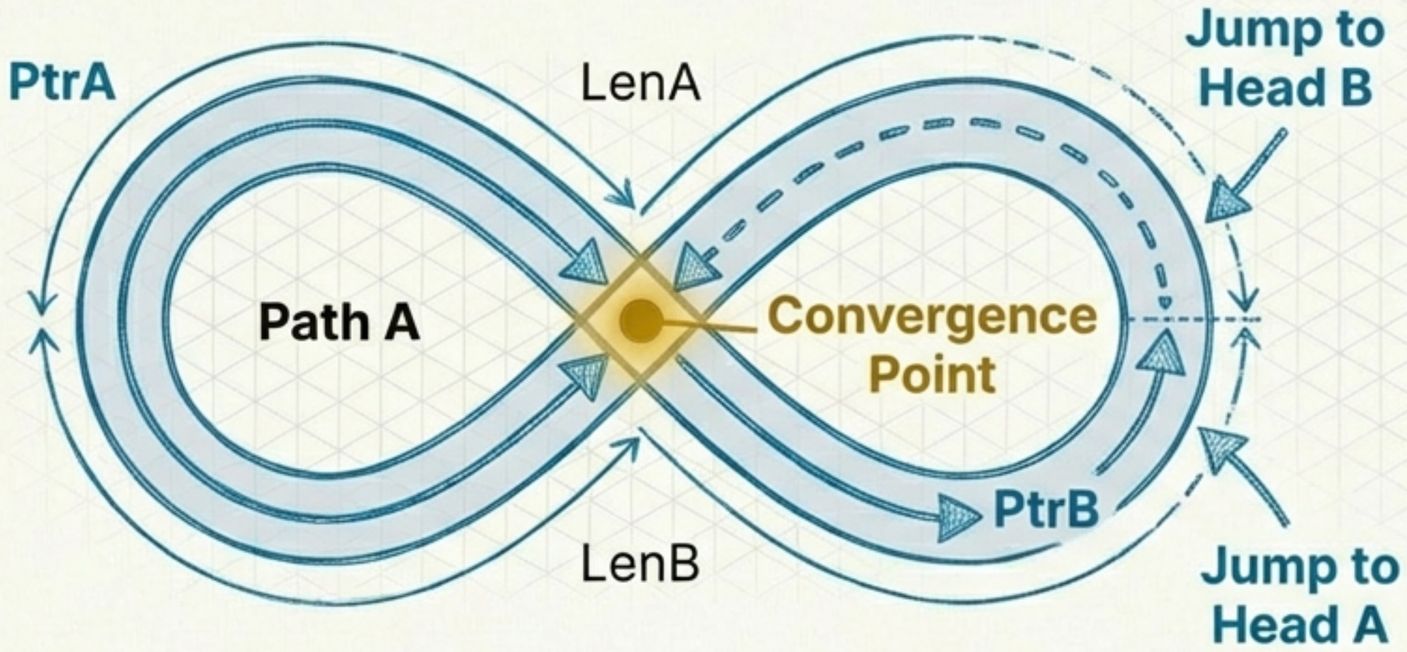


TC: O(M + N) **SC:** O(1)

Note: Optimal, but requires verbose code (two passes for length).

The Optimal Pattern: Pointer Swap

Equalizing distance without counting.




```
PtrA Dist = LenA + LenB
PtrB Dist = LenB + LenA

Therefore: A + B = B + A
```

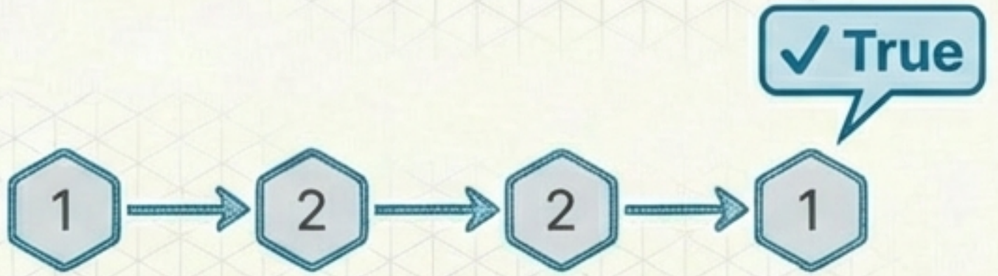
1. Initialize Pointers at Heads.
2. If PtrA hits null → Jump to Head B.
3. If PtrB hits null → Jump to Head A.

TC: O(M + N) | **SC:** O(1) | **Code:** Elegant

The Mirror Check (LeetCode 234)

Problem Definition

Challenge: Determine if a Singly Linked List reads the same forward and backward.



The “Cheat” Solution

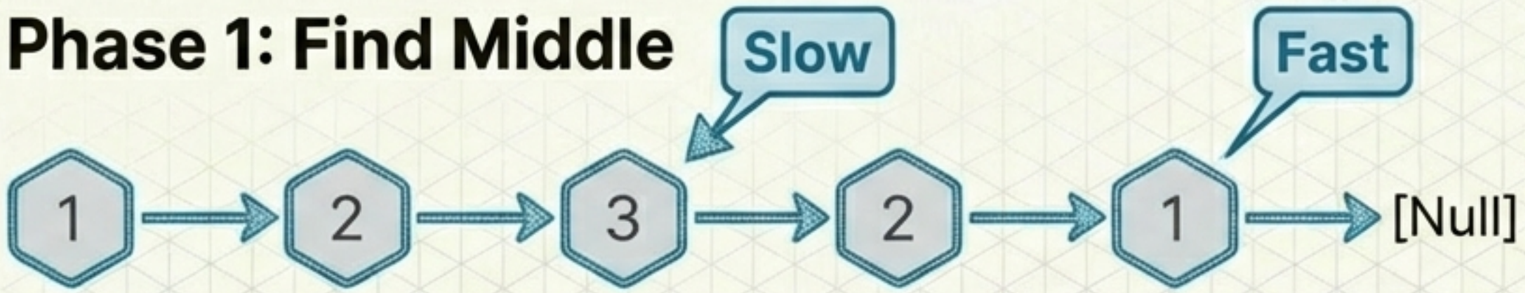
The Easy Way Out (ArrayList Approach)

Copying data to an array solves the problem but cheats the memory requirement.

Anatomy of an In-Place Solution

Modifying the structure to verify symmetry.

Phase 1: Find Middle



Slow/Fast Pattern finds the midpoint.

Phase 2: Reverse Second Half

Reversing the second half for comparison.

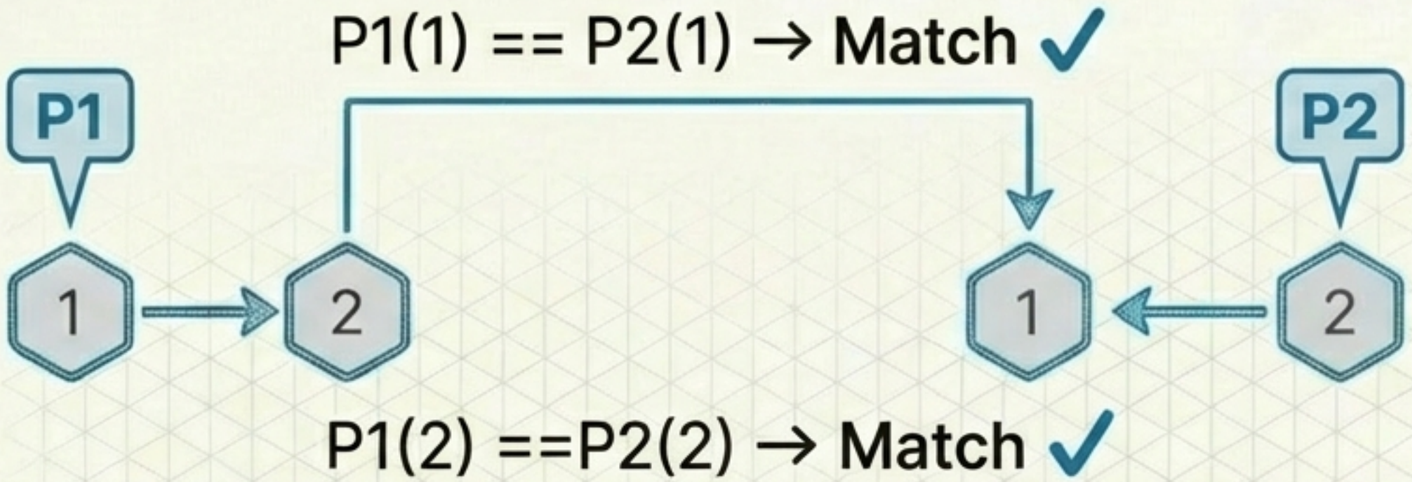
Code Logic

Step 1: Move Fast 2x, Slow 1x.

Step 2: Reverse linked list starting from Slow.next.

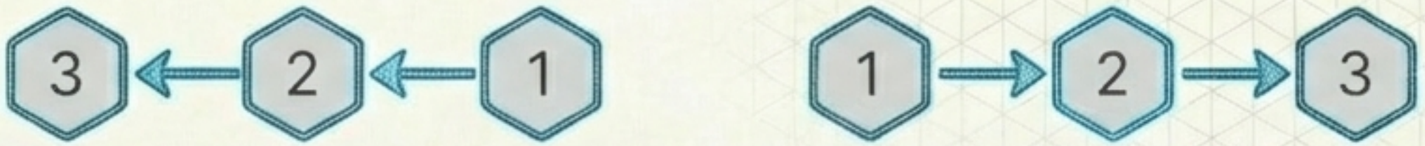
Note: This is O(1) space because we are re-wiring existing nodes.

Compare & Restore



P1(1) == P2(2) → Match ✓

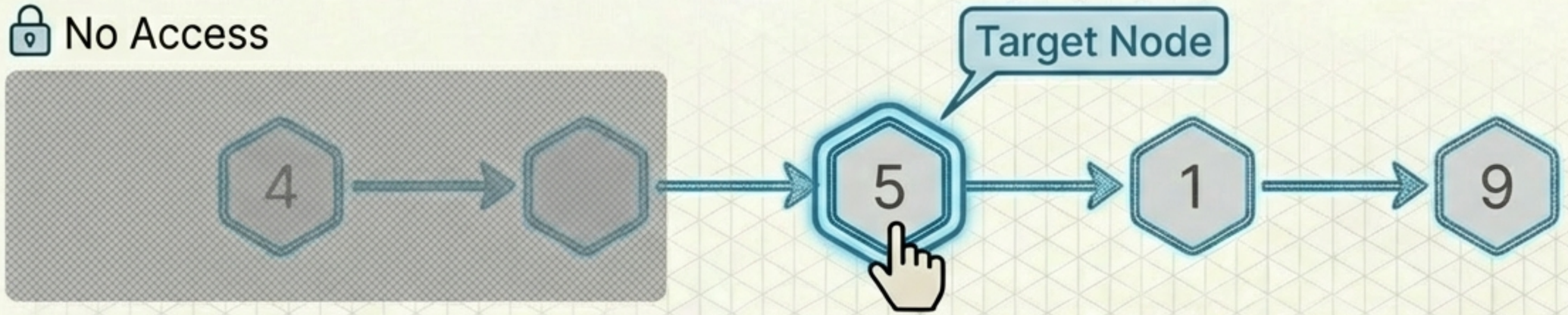
Engineering Manners: Restore the List



Always reverse the second half back to its original state before returning. Don't leave the input broken.

Final Stats:
TC: O(N) | SC: O(1)

The Impossible Deletion (LeetCode 237)



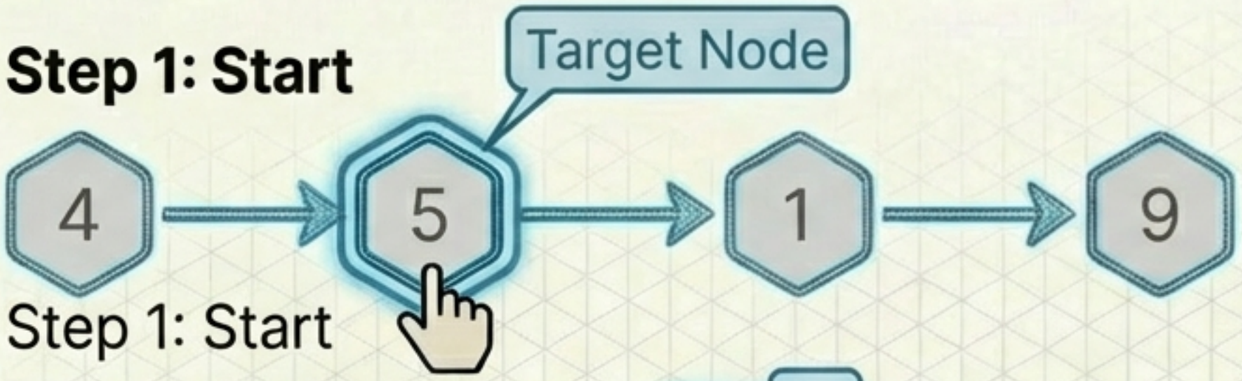
The Dilemma

- **Problem:** Delete node [5].
- **Constraint:** You do not have access to the Head. You cannot iterate to find the Previous node.
- **Standard Deletion:** `prev.next = target.next` (Impossible here).

The Data Masquerade

If you can’t delete the node, change its identity.

Step 1: Start

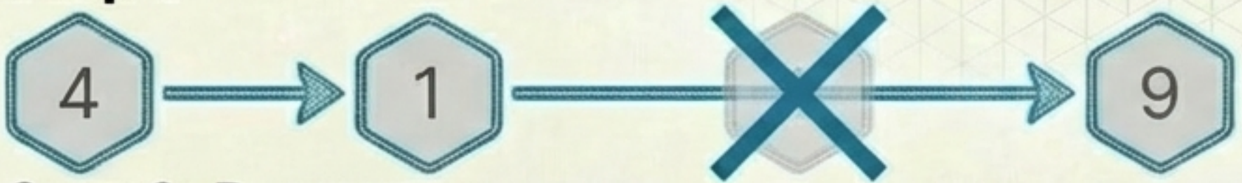


Step 1: Start

Step 2: Copy Value

Step 2: Copy Value

Step 3:



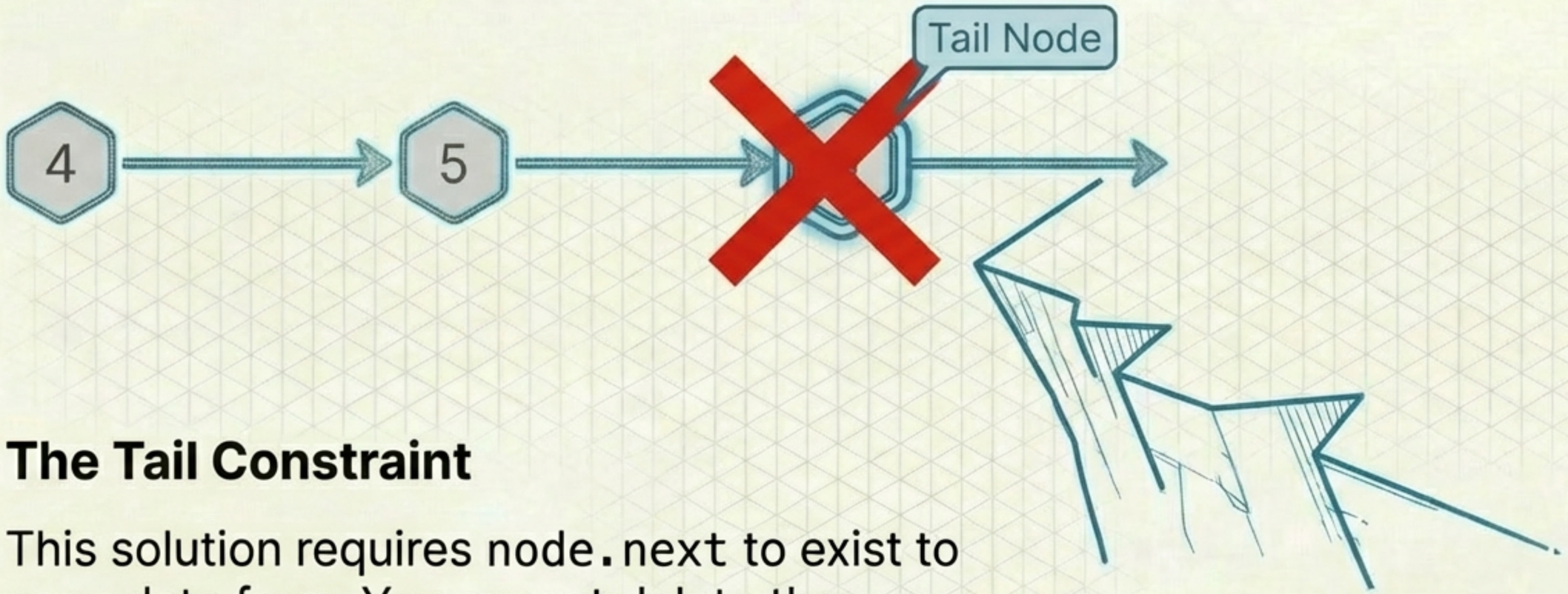
Step 3: Bypass

```
node.val = node.next.val
```

```
node.next = node.next.next
```

Final Stats:
TC: O(1) | SC: O(1)

Limitations of the Trick



The Tail Constraint

This solution requires `node.next` to exist to copy data from. You cannot delete the Tail node using this method because there is no successor to mimic.

Source Note: LeetCode 237 guarantees the target is not the tail. In real systems, this edge case requires a doubly linked list or head access.

The Optimization Toolkit

Pointer Swap Math

Equalize travel distances by swapping heads.

Slow & Fast Pointers

Find midpoints or cycles in a single pass.

In-Place Modification

Reverse links to verify state without O(N) space.

Value Overwriting

Manipulate data when structural access is denied.

Find the Pattern

Linked List problems are rarely about the data. They are tests of of your ability to manipulate references and visualize memory.

If the solution feels too **complex**,
you are likely missing the **trick**.

Content adapted from ‘CTO Bhaiya’ technical walkthroughs.